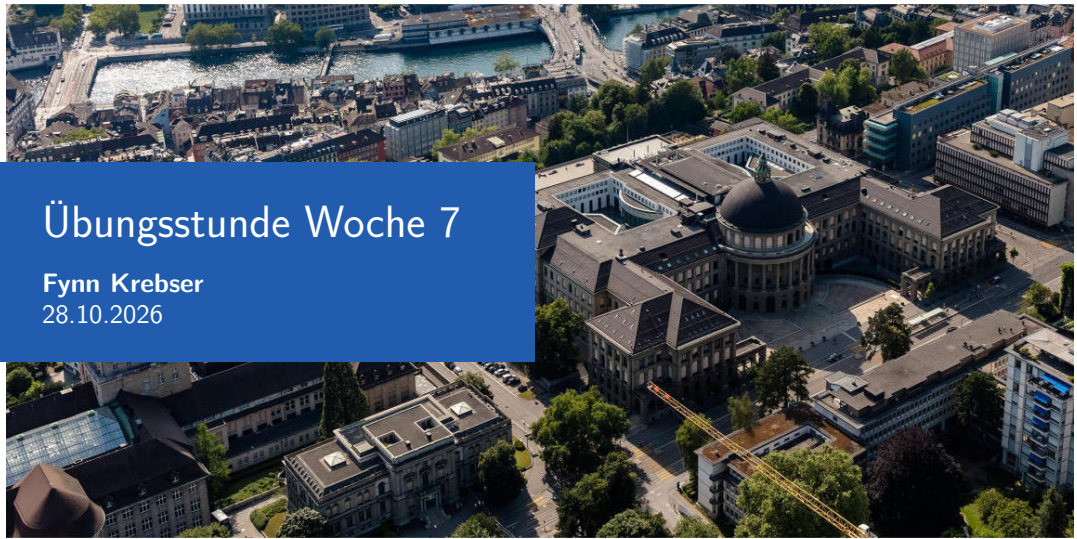
ETH zürich



Übungsstunde Woche 7

Fynn Krebser

28.10.2026

50:50 Pub Tel

Frage 1 ▪ 50 €

Was wird durch `std::cout << 'a'- 'A';` auf der Konsole ausgegeben?

A: Ein Leerzeichen

B: 32

C: 'a'

D: Kompilierfehler

15	1 000 000 €
14	500 000 €
13	125 000 €
12	64 000 €
11	32 000 €
10	16 000 €
9	8 000 €
8	4 000 €
7	2 000 €
6	1 000 €
5	500 €
4	300 €
3	200 €
2	100 €
1	50 €

50:50 Pub Tel

Frage 2 ▪ 100 €

Wie wandelt man eine Variable `char c = 'G' ;` korrekt in ihren Kleinbuchstaben `'g'` um?

A: `c + 1`

B: `c - 'a' + 'A'`

C: `c - 'A' + 'a'`

D: `c * 2`

15	1 000 000 €
14	500 000 €
13	125 000 €
12	64 000 €
11	32 000 €
10	16 000 €
9	8 000 €
8	4 000 €
7	2 000 €
6	1 000 €
5	500 €
4	300 €
3	200 €
2	100 €
1	50 €

50:50 Pub Tel

Frage 3 ▪ 200 €

Was ist das Ergebnis von `std::string s = "ETH"; s += "Zurich";`?

A: `s` ist `"ETH Zurich"`

B: Kompilierfehler

C: `s` ist `" Zurich"`

D: Undefiniertes Verhalten

15	1 000 000 €
14	500 000 €
13	125 000 €
12	64 000 €
11	32 000 €
10	16 000 €
9	8 000 €
8	4 000 €
7	2 000 €
6	1 000 €
5	500 €
4	300 €
3	200 €
2	100 €
1	50 €

50:50 Pub Tel

Frage 4 ▪ 300 €

Was passiert zur Laufzeit, wenn bei einer rekursiven Funktion die Abbruchbedingung fehlt?

A: Sie bricht nach 100 Aufrufen automatisch ab

B: Stack Overflow / Speicherüberlauf

C: Der Compiler verweigert das Kompilieren

D: Die Funktion gibt automatisch 0 zurück

15	1 000 000 €
14	500 000 €
13	125 000 €
12	64 000 €
11	32 000 €
10	16 000 €
9	8 000 €
8	4 000 €
7	2 000 €
6	1 000 €
5	500 €
4	300 €
3	200 €
2	100 €
1	50 €

Serie letzter Woche

- **Const and Reference Types**: Interessant ist hauptsächlich folgende Funktion

```
const int& foo(int& i) {
    return ++i;
}
```

- **Number of Occurences**: Für die Fortgeschrittenen: Const-Korrektheit
- **Sorting by Swapping**: Sehr gut gelöst.
- **Vector and Matrix Operations**: Wenige Abgaben, wenn ihr mal Zeit habt, könnt ihr das gerne noch nachreichen :)

Fragen?

Ziel von Heute

- ASCII und `char`
- Rekursion

Datentyp: `char`

- Datentyp welcher Buchstaben, Zahlen oder Sonderzeichen speichert.
- In C++ werden `char` als 8-Bit-Zahlen gespeichert.
- Konvertierung nach dem ASCII-Standard.

Dezimal	Zeichen	Dezimal	Zeichen	Dezimal	Zeichen
32	*(Leerzeichen)*	65	A	97	a
48	0	66	B	98	b
49	1	67	C	99	c
...	...	...	...	...	...
57	9	90	Z	122	z

Rechnen mit Chars

Da die ASCII-Zeichen als Zahlen gespeichert werden, können wir auch mit ihnen rechnen.

- `'A'` + 1 ergibt `'B'`
- `'a'` - `'A'` ergibt 32.
- `'0'` + 5 ergibt `'5'`.
- `'G'` - `'A'`+ `'a'` ergibt `'g'`.

Aufgepasst!

Bei Rechnungen mit `char` wird das Resultat als `int` zurückgegeben. `char c = 'a'- 'A';` funktioniert, `std::cout << 'a'- 'A';` gibt aber 32 aus, nicht `' '`!

[code]expert: Converting to UPPER CASE

- [code]expert: Converting to UPPER CASE

Strings

- Ein String ist eine Art Vektor von `char`.
- Folglich ist die Verwendung sehr ähnlich, z.B. `std::string s = "Hello";` und `s[0]` gibt `'H'` zurück.
- Strings haben auch eine Länge, die mit `std::ssize(s)` abgefragt werden kann.
- Strings können mit + oder += zusammengefügt werden. (z.B. `std::string s = "Hello"; s += "World";` ergibt `s == "Hello World"`).

Rekursion

Um Rekursion zu verstehen, muss man Rekursion verstehen. :)



Rekursion

- Rekursion ist ein Programmierkonzept, bei dem eine Funktion sich selbst aufruft.
- Um die n-te Fibonacci-Zahl zu berechnen, muss man die $(n-1)$-te und $(n-2)$-te Fibonacci-Zahl addieren. Woher bekommen wir diese Zahlen? Wir rufen die Funktion einfach erneut auf, um sie zu berechnen.
- Jede rekursive Funktion muss eine Abbruchbedingung haben, um unendliche Rekursion zu vermeiden.

Genereller Ansatz: Überlegt was ihr alles braucht um euer Resultat zu berechnen. Häufig braucht ihr dafür das Resultat von kleineren Teilproblemen. Diese könnt ihr dann einfach durch einen rekursiven Funktionsaufruf berechnen.

[code]expert: Computing Series

- [code]expert: Computing Series

Rekursionen Üben

Auf Code-Expert gibt es unter den Code Examples noch folgende Beispiele[1]

- Partial Sum
- Power Function

Wenn ihr fragen habt, könnt ihr mich gerne kontaktieren.
Weitere Übungsmöglichkeiten: Fragt KI nach einer Rekursiven Funktionsdefinition, implementiert diese und vergleicht das Resultat für verschiedene Inputs mit der KI.

Als fortgeschrittenes Beispiel gibt es auf CodeExpert noch "The Towers of Hanoi"

[1] Sind beide nicht extrem gut meiner Meinung nach.

Tipps zu den Übungen

Recursive Function Analysis

- Zeichnet in einem Diagramm auf wie sich die Funktionsaufrufe aufbauen.

Bitstrings up to n

- Modifiziert den Algorithmus aus der Vorlesung für alle Bitstrings der Länge n.